

Kubernetes JOBS

What are Kubernetes Jobs?

Faculty Definition:

Kubernetes Jobs are resources used to run batch tasks in a Kubernetes cluster. They ensure that a specified number of Pods successfully complete a task before marking the Job as done.

Simple Meaning:

- Jobs are used for temporary tasks.
 - Once the task completes successfully, the Job stops.
-

Real-Time Use Cases of Jobs

Jobs are mainly used for:

- Database backup
 - Data migration
 - Batch processing
 - Sending emails
 - Report generation
 - One-time scripts
 - Scheduled tasks
-

Important Point

Unlike Deployments:

- Jobs are NOT for continuously running applications.

Jobs are:

One-time execution tasks.

Types of Kubernetes Jobs

1. Non-Parallel Jobs

Faculty Definition:

These Jobs execute tasks sequentially / one by one, with only one Pod started unless it fails.

Simple Meaning:

- Only one Pod runs at a time.
- After completion, Job ends.

2. Parallel Jobs with Fixed Completion Count

Faculty Definition:

Multiple Pods run simultaneously to complete a task.
The Job is complete when specified number of Pods complete successfully.

Simple Meaning:

- Multiple Pods work together.
- Job finishes after required successful completions.

3. Parallel Jobs with Work Queue

Faculty Definition:

These Jobs dynamically scale Pods based on workload.
Pods coordinate with each other or external services to fetch and process tasks from shared queue.

Simple Meaning:

- Pods take tasks from queue.

- Useful for heavy workloads.
-

Visual Understanding of Jobs

Non-Parallel Job

Image Meaning:

- Completion = 4
- Pods run one by one sequentially.

Pod1 → complete

Pod2 → complete

Pod3 → complete

Pod4 → complete

Only one Pod runs at a time.

Parallel Job

Image Meaning:

- completion = 4
- parallelism = 2

Two Pods run together simultaneously.

Example:

Pod1 + Pod2 run together

then

Pod3 + Pod4 run together

This reduces execution time.

Important Terms

completion

completions: 3

Faculty Definition:

The Job completes only when 3 Pods have successfully run.

Simple Meaning:

- Total successful runs required.
-

parallelism

parallelism: 2

Meaning:

- Two Pods can run at same time.
-

BusyBox Explanation

Faculty Definition:

BusyBox is a single executable that bundles multiple Unix utilities like sh, ls, cat, echo, wget, grep etc.

Simple Meaning:

- BusyBox is a lightweight Linux image.
- Mostly used in Kubernetes examples and Jobs.

Advantages:

- Very small size
 - Fast startup
 - Simple utility commands
-

restartPolicy

Kubernetes supports:

Always
OnFailure
Never

restartPolicy: Always

Faculty Definition:

Always restarts the container if it exits.

Use Case:

Deployments & ReplicaSets

Simple Meaning:

- Container restarts continuously.

Used in:

- Web applications
 - Long-running apps
-

restartPolicy: OnFailure

Faculty Definition:

Restarts container only if it exits with error.

Use Case:

Jobs & CronJobs

Simple Meaning:

- Restart only when failure happens.
-

restartPolicy: Never

Faculty Definition:

Never restarts the container even if it fails.

Use Case:

Jobs & CronJobs

Simple Meaning:

- Run once only.

Mostly used for:

One-time execution jobs

Non-Parallel Job YAML

apiVersion: batch/v1

kind: Job

metadata:

name: non-parallel-job

spec:

template:

metadata:

name: non-parallel-pod

spec:

containers:

- name: non-parallel-container

image: busybox

command: ["echo", "Hello from the non-parallel job"]

restartPolicy: Never

completions: 3

YAML Explanation

apiVersion

apiVersion: batch/v1

Purpose:

- Jobs belong to batch API group.
-

kind

kind: Job

Purpose:

- Creates Kubernetes Job object.
-

metadata

metadata:

name: non-parallel-job

Purpose:

- Name of Job.
-

template

template:

Purpose:

- Defines Pod template.
-

containers

containers:

Purpose:

- Defines containers inside Pod.
-

command

command: ["echo", "Hello from the non-parallel job"]

Purpose:

- Prints message once.
-

restartPolicy

restartPolicy: Never

Purpose:

- Do not restart Pod after completion.
-

completions

completions: 3

Purpose:

- Job succeeds only after 3 successful executions.
-

Create Job

Command:

```
kubecttl apply -f nonparallel.yml
```

Verify Jobs

Command:

```
kubecttl get jobs
```

Verify Pods

Command:

```
kubecttl get pods
```

Check Logs

Command:

```
kubecttl logs <pod-name>
```

Expected Output:

```
Hello from the non-parallel job
```

Difference Between Deployment and Job

Deployment	Job
Runs continuously	Runs temporarily
Used for applications	Used for tasks

Restart always

Complete and
stop

Web apps

Batch tasks

Real-Time Example

Deployment Example

Netflix Application

Must run:

24/7 continuously

So:

Use Deployment

Job Example

Daily database backup

Runs:

One time and exits

So:

Use Job

Important Interview Question

Question:

Why do we use Kubernetes Jobs?

Answer:

Jobs are used for batch processing and one-time tasks.
They ensure Pods complete successfully before marking task as complete.

Important Interview Question

Question:

What is the difference between completion and parallelism?

Answer:

completions

Total successful Pods required.

parallelism

How many Pods can run simultaneously.

Important Interview Question

Question:

Why use restartPolicy: Never in Jobs?

Answer:

Because Jobs are one-time execution tasks.
After completion, Pod should stop instead of restarting.

Important Interview Question

Question:

Why is BusyBox commonly used in Jobs?

Answer:

BusyBox is lightweight, fast, and contains useful Linux utilities.

Final Quick Revision

Kubernetes Jobs:

- Used for temporary tasks.

Types:

1. Non-parallel Job
2. Parallel Job with fixed completion
3. Parallel Job with work queue

Important fields:

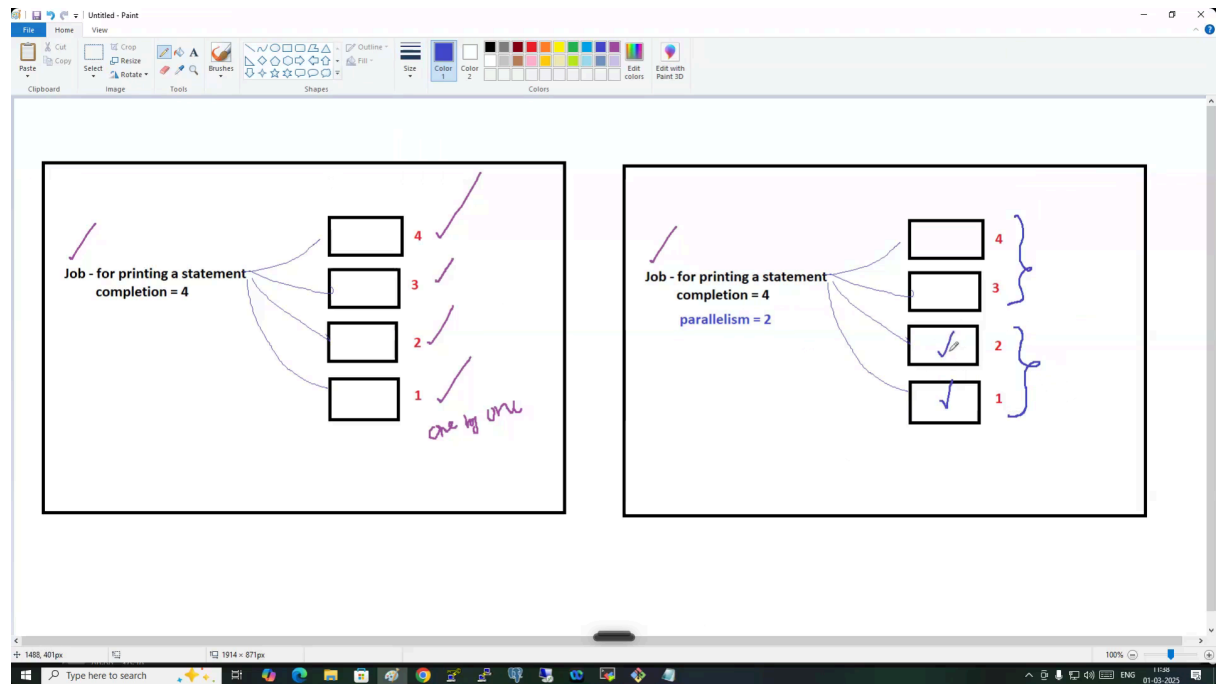
- completions
- parallelism
- restartPolicy

restartPolicy:

- Always
- OnFailure
- Never

Jobs are used for:

- backups
- migrations
- batch processing
- scheduled tasks



Your screenshot contains a Kubernetes Job manifest (`kind: Job`).

Here is the corrected and executable Job manifest.

Correct `nonparallel.yml`

apiVersion: batch/v1

kind: Job

metadata:

name: non-parallel-job

spec:

completions: 3

template:

metadata:

name: non-parallel-pod

spec:

containers:

- name: non-parallel-container

image: busybox

command:

- "echo"

- "Hello from the non-parallel job"

restartPolicy: Never

Copy-Paste Commands

Step 1 — Create YAML File

vi nonparallel.yml

Paste the YAML above.

Save file:

ESC → :wq

Step 2 — Create Job

```
kubectl apply -f nonparallel.yml
```

Step 3 — Verify Job

```
kubectl get jobs
```

Expected output:

NAME	COMPLETIONS	DURATION	AGE
non-parallel-job	3/3	20s	25s

Step 4 — Check Pods Created by Job

```
kubectl get pods
```

You may see completed pods.

Step 5 — View Job Details

```
kubectl describe job non-parallel-job
```

Step 6 — View Logs

Get pod names:

```
kubectl get pods
```

Then check logs:

```
kubectl logs <pod-name>
```

Step 7 — Delete Job

```
kubectl delete -f nonparallel.yml
```

or

```
kubectl delete job non-parallel-job
```

Important Beginner Understanding

Kubernetes Job

A Job:

- runs a task until completion
 - unlike Deployment, it does NOT run forever
 - used for:
 - backup jobs
 - scripts
 - batch processing
 - migrations
-

Meaning of **completions: 3**

completions: 3

The Job succeeds only after:

- 3 pods complete successfully
-

Meaning of **restartPolicy: Never**

restartPolicy: Never

If container exits:

- Kubernetes will NOT restart container inside same pod
- Instead, Job controller may create new pod if needed

About BusyBox

busybox is:

- lightweight Linux container
- commonly used for testing
- contains basic Linux commands:
 - sh
 - echo
 - ls
 - cat

In your screenshot, `completions` and `parallelism` are written after the pod template block inside the main Job `spec:` block.

Here is the corrected version in the same style as your terminal:

```
apiVersion: batch/v1
```

```
kind: Job
```

```
metadata:
```

```
  name: non-parallel-job
```

```
spec:
```

```
  template:
```

```
    metadata:
```

```
      name: non-parallel-pod
```

```
    spec:
```

```
      containers:
```

```
        - name: non-parallel-container
```

```
          image: busybox
```

```
          command: ["echo", "Hello from the non-parallel job"]
```

```
      restartPolicy: Never
```

```
completions: 6
```

```
parallelism: 3
```

This YAML is VALID because:

- `template`
- `completions`
- `parallelism`

all are at the same indentation level under the main Job `spec:`.

Create File

`vi parallel-job.yml`

Paste YAML and save:

ESC → `:wq`

Apply Job

`kubectl apply -f parallel-job.yml`

Verify

`kubectl get jobs`

Watch Pods

`kubectl get pods -w`

Your screenshots contain a Kubernetes `CronJob` manifest with notes mixed into YAML.

I corrected the formatting, indentation, and removed invalid inline explanation text.

Correct `cron.yml`

`apiVersion: batch/v1`

kind: CronJob

metadata:

name: hello-cronjob

spec:

schedule: "*/1 * * * *

jobTemplate:

spec:

template:

spec:

containers:

- name: hello-container

image: busybox

command:

- "sh"

- "-c"

- "echo Hello from Kubernetes!"

restartPolicy: Never

What This CronJob Does

This CronJob:

- runs every 1 minute
- creates a Job every minute
- Job creates a Pod
- Pod prints:

Hello from Kubernetes!

Copy-Paste Commands

Step 1 — Create File

vi cron.yml

Paste YAML and save:

ESC → :wq

Step 2 — Apply CronJob

kubectl apply -f cron.yml

Step 3 — Verify CronJob

kubectl get cronjobs

Expected output:

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE
hello-cronjob	*/* * * * *	False	0	1m

Step 4 — Check Jobs Created by CronJob

kubectl get jobs

Step 5 — Check Pods

kubectl get pods

You will see completed pods like:

hello-cronjob-29013511-ht5zc

Step 6 — View Logs

kubectl logs <pod-name>

Example:

```
kubectl logs hello-cronjob-29013511-ht5zc
```

Expected output:

Hello from Kubernetes!

Step 7 — Describe CronJob

```
kubectl describe cronjob hello-cronjob
```

Step 8 — Delete CronJob

```
kubectl delete -f cron.yml
```

or

```
kubectl delete cronjob hello-cronjob
```

Important Beginner Understanding

CronJob

A CronJob:

- runs tasks automatically on schedule
- similar to Linux cron

Used for:

- backups
 - cleanup tasks
 - reports
 - automation scripts
-

Meaning of Schedule

schedule: `"*/1 * * * *"`

Means:

- every 1 minute
-

Cron Format

`* * * * *`

